

DATA-260 Homework 5
Rental Housing Listings Application

Student: Xuanhua Li
SID4: 2974
Domain: Rental Housing Listings
Submission Tag: hw5

GitHub Repository:
<https://github.com/carrovo/data260-2974.git>

Environment and Configuration

Hardware Model: MacBook Air (Mac16,13)
Chip: Apple M4
CPU Cores: 10 (4 performance and 6 efficiency)
Memory: 16 GB
Operating System: macOS 15.6 (Build 24G84)

Python: 3.12.14
Node.js: 25.8.0
npm: 11.11.0
MySQL: 9.6.0
Ollama: 0.33.2
Local Model: qwen3:8b

PORT_BASE: 8274
PREFIX: s2974
SEED: 2974
VERIFY_SEED: 262974
DOMAIN_ID: 6

Implementation Commit:
d3450e1c30d452475ef136accfe5141df3a80bef

Part 1

This homework builds on HW4. Your existing FastAPI + MySQL backend gets extended with a second entity type that has a meaningful relationship to your primary domain entity - keep extending your existing service, don't start a new one.

I. Database

- Create a second table for a related entity type (e.g., for a book/author-style setup, this second entity plays the role of the "author"). It should include id (primary key), a primary text field, a secondary text field, a unique field (e.g., email or code), and timestamps.
- Your primary domain entity's table should contain id (primary key), primary field, a unique field (e.g., an ID/code), a numeric field with a sensible default (e.g., an available-count style field), a foreign key to the related entity, and timestamps. Use appropriate data types and constraints.
- Do not store plain-text passwords. Prevent deletion of a related-entity record that still has associated primary-entity records, unless you deliberately implement and document cascade behavior.

```
❏ ((.venv) ) carol@Carols-MacBook-Air data260-2974 % PYTHONPATH=code python - <<'PY'
from sqlalchemy import inspect
from database import db_session_base26

inspector = inspect(db_session_base26)

print("Student: Xuanhua Li | SID4: 2974")
print("\nDATABASE TABLES")
for table in inspector.get_table_names():
    print(f"- {table}")

for table in ["property_managers", "listings"]:
    print(f"\n{table.upper()} COLUMNS")
    print(f"{'COLUMN':24} {'TYPE':22} {'NULLABLE':22}")
    print("-" * 58)
    for column in inspector.get_columns(table):
        print(
            f"{'column['name']:24} "
            f"{'str(column['type']):22} "
            f"{'column['nullable']:'}"
        )
    )
PY
Student: Xuanhua Li | SID4: 2974

DATABASE TABLES
- listing_events
- listings
- property_managers
- sessions
- users

PROPERTY MANAGERS COLUMNS
COLUMN                                TYPE                                NULLABLE
-----
id                                     INTEGER                             False
firstName                             VARCHAR(100) COLLATE "utf8mb4_unicode_ci" False
lastName                              VARCHAR(100) COLLATE "utf8mb4_unicode_ci" False
email                                 VARCHAR(255) COLLATE "utf8mb4_unicode_ci" False
created_at                            DATETIME                             False
updated_at                            DATETIME                             False

LISTINGS COLUMNS
COLUMN                                TYPE                                NULLABLE
-----
id                                     INTEGER                             False
listingTitle                           VARCHAR(100) COLLATE "utf8mb4_unicode_ci" False
address                               VARCHAR(200) COLLATE "utf8mb4_unicode_ci" False
submitterEmail                         VARCHAR(200) COLLATE "utf8mb4_unicode_ci" False
description                            TEXT COLLATE "utf8mb4_unicode_ci" False
propertyType                           VARCHAR(30) COLLATE "utf8mb4_unicode_ci" False
termsAccepted                          TINYINT                             False
created_at                            DATETIME                             False
listingCode                           VARCHAR(30) COLLATE "utf8mb4_unicode_ci" False
monthlyRent                           DECIMAL(10, 2)                       False
availableUnits                         INTEGER                             False
propertyManagerId                     INTEGER                             False
updated_at                            DATETIME                             False
❏ ((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % PYTHONPATH=code python - <<'PY'
from sqlalchemy import text
from database import db_session_base26

query = text("""
SELECT
    l.id AS listing_id,
    l.listingCode AS code,
    l.listingTitle AS title,
    l.monthlyRent AS rent,
    l.availableUnits AS units,
    l.propertyManagerId AS manager_id,
    CONCAT(pm.firstName, ' ', pm.lastName) AS manager
FROM listings AS l
JOIN property_managers AS pm
    ON pm.id = l.propertyManagerId
ORDER BY l.id DESC
LIMIT 5
""")

print("Student: Xuanhua Li | SID4: 2974")
print("\nLISTING -> PROPERTY MANAGER RELATIONSHIP")
print(f"{'ID':7} {'CODE':17} {'RENT':10} {'UNITS':7} {'MGR ID':8} MANAGER")
print("-" * 72)

with db_session_base26.connect() as connection:
    for row in connection.execute(query).mappings():
        print(
            f"{'row['listing_id']':<7} "
            f"{'row['code']':<17} "
            f"{'str(row['rent']):<10} "
            f"{'row['units']':<7} "
            f"{'row['manager_id']':<8} "
            f"{'row['manager']}'"
        )
PY
Student: Xuanhua Li | SID4: 2974

LISTING -> PROPERTY MANAGER RELATIONSHIP
ID      CODE          RENT      UNITS  MGR ID  MANAGER
-----
5003    S2974-005003    1650.00   1      1      Default Manager
5002    S2974-005002    1600.00   1      1      Default Manager
5001    S2974-005001    1550.00   1      1      Default Manager
5000    S2974-005000    1500.00   1      1      Default Manager
4999    S2974-004999    2450.00   1      1      Default Manager
• ((.venv) ) carol@Carols-MacBook-Air data260-2974 %

```

II. API Implementation

- Implement CRUD endpoints for both entities.
- For the related entity: add new records, retrieve all with pagination, get individual records, update, delete.
- For your primary domain entity: same set - create, list, retrieve, update, delete.
- Include one additional endpoint to get all primary-entity records associated with a specific related-entity record (the relationship query).
- Use Pydantic schemas for request/response validation. Implement proper HTTP status codes and error messages for resource-not-found, validation errors, and constraint violations. Ensure the unique field's format is validated.
- Submit Postman screenshots of each action tested using the API endpoints. Submit a screenshot of your DB

DATA260 HW5 Part 1 | POST 00 - Login | POST 01 - Create Property Manager | + | No Environment |

DATA260 HW5 Part 1 > 01 - Create Property Manager | Save | Share

POST | {{base_url}} /api/property-managers | Send

Docs | Params | Authorization | Headers 10 | Body | Scripts | Settings | Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON | Schema | Beautify

```
1 {
2   "firstName": "Postman",
3   "lastName": "Manager",
4   "email": "postman.manager.2974@example.com"
5 }
```

Body | Cookies 1 | Headers 4 | Test Results | 201 Created • 52 ms • 297 B • Save Response

{ JSON | Preview | Visualize |

```
1 {
2   "id": 4,
3   "firstName": "Postman",
4   "lastName": "Manager",
5   "email": "postman.manager.2974@example.com",
6   "created_at": "2026-10-04T11:45:19",
7   "updated_at": "2026-10-04T11:45:19"
8 }
```

DATA260 HW5 Part 1 | POST 00 - Login | POST 01 - Create Property Manager | GET 02 - List Property Manager | + | No Environment |

DATA260 HW5 Part 1 > 02 - List Property Managers | Save | Share

GET | {{base_url}} /api/property-managers?skip=0&limit=10 | Send

Docs | Params | Authorization | Headers 8 | Body | Scripts | Settings | Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body | Cookies 1 | Headers 4 | Test Results | 200 OK • 45 ms • 450 B • Save Response

{ JSON | Preview | Visualize |

```
1 [
2   {
3     "id": 1,
4     "firstName": "Default",
5     "lastName": "Manager",
6     "email": "manager2974@example.com",
7     "created_at": "2026-10-03T23:05:36",
8     "updated_at": "2026-10-03T23:05:36"
9   },
10  {
11    "id": 4,
12    "firstName": "Postman",
13    "lastName": "Manager",
14    "email": "postman.manager.2974@example.com",
15    "created_at": "2026-10-04T11:45:19",
16    "updated_at": "2026-10-04T11:45:19"
17  }
18 ]
```

DATA260 HW5 Part 1 > 03 - Get Property Manager

GET `{{base_url}}/api/property-managers/{{manager_id}}` Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies 1 Headers 4 Test Results 200 OK • 31 ms • 292 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 4,
3   "firstName": "Postman",
4   "lastName": "Manager",
5   "email": "postman.manager.2974@example.com",
6   "created_at": "2026-10-04T11:45:19",
7   "updated_at": "2026-10-04T11:45:19"
8 }
```

DATA260 HW5 Part 1 > Update Property Manager

PUT `{{base_url}}/api/property-managers/{{manager_id}}` Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "firstName": "Postman",
3   "lastName": "Manager Updated",
4   "email": "postman.manager.2974@example.com"
5 }
```

Body Cookies 1 Headers 4 Test Results 200 OK • 43 ms • 300 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 4,
3   "firstName": "Postman",
4   "lastName": "Manager Updated",
5   "email": "postman.manager.2974@example.com",
6   "created_at": "2026-10-04T11:45:19",
7   "updated_at": "2026-10-04T12:01:09"
8 }
```

DATA260 | POST 00 - Log | POST 01 - Cre | GET 02 - List | GET 03 - Get | PUT Update P | **POST 05 - Duplicate Manager Email** | + | No Environment |

DATA260 HW5 Part 1 > **05 - Duplicate Manager Email** | Save | Share

POST | `{{base_url}}/api/property-managers` | Send

Docs | Params | Authorization | Headers 10 | **Body** | Scripts | Settings | Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON | Schema | Beautify

```
1 {
2   "firstName": "Duplicate",
3   "lastName": "Manager",
4   "email": "postman.manager.2974@example.com"
5 }
```

Body | Cookies 1 | Headers 4 | Test Results | 409 Conflict • 41 ms • 183 B • Save Response

✖ Resolve duplicate email error | Diagnose | Decline

{ } JSON | Preview | Debug with AI

```
1 {
2   "detail": "Property manager email must be unique"
3 }
```

DATA260 | POST 00 - L | POST 01 - C | GET 02 - L | GET 03 - Ge | PUT 04 - Up | POST 05 - • | **POST 06 - Invalid Manager Email** | + | No Environment |

DATA260 HW5 Part 1 > **06 - Invalid Manager Email** | Save | Share

POST | `{{base_url}}/api/property-managers` | Send

Docs | Params | Authorization | Headers 10 | **Body** | Scripts | Settings | Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON | Schema | Beautify

```
1 {
2   "firstName": "Invalid",
3   "lastName": "Email",
4   "email": "not-an-email"
5 }
```

Body | 422 Unprocessable Entity (WebDAV) (RFC 4918) • 27 ms • 366 B • Save Response

✖ Investigate test case requirements | Diagnose | Decline

{ } JSON | Preview | Debug with AI

```
1 {
2   "detail": [
3     {
4       "type": "value_error",
5       "loc": [
6         "body",
7         "email"
8       ],
9       "msg": "value is not a valid email address: An email address must have an @-sign.",
10      "input": "not-an-email",
11      "ctx": {
12        "reason": "An email address must have an @-sign."
13      }
14    }
15  ]
16 }
```

DATA260 HW5 Part 1 > 07 - Manager Not Found

GET `{{(base_url)}}/api/property-managers/999999` Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies 1 Headers 4 Test Results 404 Not Found 59 ms 173 B Save Response

Handle property manager not found Diagnose Decline

`{}` JSON Preview Debug with AI

```
1 {
2   "detail": "Property manager not found"
3 }
```

DATA260 HW5 Part 1 > 08 - Create Listing

POST `{{(base_url)}}/api/listings` Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "listingTitle": "Postman Test Apartment",
3   "listingCode": "S2974-POSTMAN-01",
4   "address": "400 API Avenue, San Jose, CA",
5   "submitterEmail": "xuanhua.hw5.2974@example.com",
6   "description": "A rental listing created for the HW5 Postman CRUD evidence.",
7   "propertyType": "Apartment",
8   "monthlyRent": 2850.00,
9   "availableUnits": 1,
10  "termsAccepted": true,
11  "propertyManagerId": "{{manager_id}}"
12 }
```

Body Cookies 1 Headers 4 Test Results 201 Created 49 ms 577 B Save Response

`{}` JSON Preview Visualize

```
1 {
2   "id": 5007,
3   "listingTitle": "Postman Test Apartment",
4   "listingCode": "S2974-POSTMAN-01",
5   "address": "400 API Avenue, San Jose, CA",
6   "submitterEmail": "xuanhua.hw5.2974@example.com",
7   "description": "A rental listing created for the HW5 Postman CRUD evidence.",
8   "propertyType": "Apartment",
9   "monthlyRent": "2850.00",
10  "availableUnits": 1,
11  "termsAccepted": true,
12  "propertyManagerId": 4,
13  "created_at": "2026-10-04T12:15:15",
14  "updated_at": "2026-10-04T12:15:15",
15  "events": []
16 }
```

GET 02 - LiGET 03 - GPUT 04 - UPOST 05 - IPOST 06 - IGET 07 - MPOST 08 - GET 09 - Li+No Environment

DATA260 HW5 Part 109 - List ListingsSaveShare

GET{{base_url}}/api/listings?skip=5000&page_size=20Send

DocsParamsAuthorizationHeaders 8BodyScriptsSettingsCookies

noneform-datax-www-form-urlencodedrawbinaryGraphQL

This request does not have a body

BodyCookies 1Headers 5Test Results200 OK49 ms595 BSave Response

JSONPreviewVisualize

```
1 [
2   {
3     "id": 5007,
4     "listingTitle": "Postman Test Apartment",
5     "listingCode": "S2974-POSTMAN-01",
6     "address": "400 API Avenue, San Jose, CA",
7     "submitterEmail": "xuanhua.hw5.2974@example.com",
8     "description": "A rental listing created for the HW5 Postman CRUD evidence.",
9     "propertyType": "Apartment",
10    "monthlyRent": "2850.00",
11    "availableUnits": 1,
12    "termsAccepted": true,
13    "propertyManagerId": 4,
14    "created_at": "2026-10-04T12:15:15",
15    "updated_at": "2026-10-04T12:15:15",
16    "events": []
17  }
18 ]
```

LiGET 03 - GPUT 04 - UPOST 05 - IPOST 06 - IGET 07 - MPOST 08 - GET 09 - LiGET 10 - Ge+No Environment

DATA260 HW5 Part 110 - Get ListingSaveShare

GET{{base_url}}/api/listings/{{listing_id}}Send

DocsParamsAuthorizationHeaders 8BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

BodyCookies 1Headers 4Test Results200 OK27 ms572 BSave Response

JSONPreviewVisualize

```
1 {
2   "id": 5007,
3   "listingTitle": "Postman Test Apartment",
4   "listingCode": "S2974-POSTMAN-01",
5   "address": "400 API Avenue, San Jose, CA",
6   "submitterEmail": "xuanhua.hw5.2974@example.com",
7   "description": "A rental listing created for the HW5 Postman CRUD evidence.",
8   "propertyType": "Apartment",
9   "monthlyRent": "2850.00",
10  "availableUnits": 1,
11  "termsAccepted": true,
12  "propertyManagerId": 4,
13  "created_at": "2026-10-04T12:15:15",
14  "updated_at": "2026-10-04T12:15:15",
15  "events": []
16 }
```

DATA260 HW5 Part 1 > 11 - Update Listing

PUT `{{(base_url)}}/api/listings/{{(listing_id)}}` Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "listingTitle": "Updated Postman Apartment",
3   "monthlyRent": 2600.00,
4   "availableUnits": 0
5 }
```

Body Cookies 1 Headers 4 Test Results 200 OK • 42 ms • 575 B Save Response

`{}` JSON Preview Visualize

```
1 {
2   "id": 5007,
3   "listingTitle": "Updated Postman Apartment",
4   "listingCode": "S2974-POSTMAN-01",
5   "address": "400 API Avenue, San Jose, CA",
6   "submitterEmail": "xuanhua.hw5.2974@example.com",
7   "description": "A rental listing created for the HW5 Postman CRUD evidence.",
8   "propertyType": "Apartment",
9   "monthlyRent": "2600.00",
10  "availableUnits": 0,
11  "termsAccepted": true,
12  "propertyManagerId": 4,
13  "created_at": "2026-10-04T12:15:15",
14  "updated_at": "2026-10-04T12:18:55",
15  "events": []
16 }
```

DATA260 HW5 Part 1 > 12 - Listings by Property Manager

GET `{{(base_url)}}/api/property-managers/{{(manager_id)}}/listings?skip=0&limit=10` Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies 1 Headers 4 Test Results 200 OK • 26 ms • 577 B Save Response

`{}` JSON Preview Visualize

```
1 [
2   {
3     "id": 5007,
4     "listingTitle": "Updated Postman Apartment",
5     "listingCode": "S2974-POSTMAN-01",
6     "address": "400 API Avenue, San Jose, CA",
7     "submitterEmail": "xuanhua.hw5.2974@example.com",
8     "description": "A rental listing created for the HW5 Postman CRUD evidence.",
9     "propertyType": "Apartment",
10    "monthlyRent": "2600.00",
11    "availableUnits": 0,
12    "termsAccepted": true,
13    "propertyManagerId": 4,
14    "created_at": "2026-10-04T12:15:15",
15    "updated_at": "2026-10-04T12:18:55",
16    "events": []
17  }
18 ]
```

DATA260 HW5 Part 1 > 13 - Duplicate Listing Code

Save

Share

POST

{{base_url}}/api/listings

Send

Docs

Params

Authorization

Headers 10

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

```
1 {
2   "listingTitle": "Duplicate Code Test",
3   "listingCode": "S2974-POSTMAN-01",
4   "address": "401 API Avenue, San Jose, CA",
5   "description": "This request intentionally tests the unique listing code constraint.",
6   "propertyType": "Apartment",
7   "monthlyRent": 2500.00,
8   "availableUnits": 1,
9   "propertyManagerId": {{manager_id}}
10 }
```

Body

Cookies 1

Headers 4

Test Results

409 Conflict

28 ms

173 B

Save Response

Handle duplicate listing code

Diagnose

Decline

JSON

Preview

Debug with AI

```
1 {
2   "detail": "Listing code must be unique"
3 }
```

DATA260 HW5 Part 1 > 14 - Blocked Manager Delete

Save

Share

DELETE

{{base_url}}/api/property-managers/{{manager_id}}

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	
Key	Value	Description		

Body

Cookies 1

Headers 4

Test Results

409 Conflict

34 ms

210 B

Save Response

Handle blocked deletion

Diagnose

Decline

JSON

Preview

Debug with AI

```
1 {
2   "detail": "Cannot delete a property manager with associated rental listings"
3 }
```

The image displays two screenshots of a REST client interface, likely Postman, showing the configuration and results of DELETE requests.

Top Screenshot: 15 - Delete Listing

- Method:** DELETE
- URL:** `{{base_url}}/api/listings/{{listing_id}}`
- Body:** none (selected). Other options: form-data, x-www-form-urlencoded, raw, binary, GraphQL.
- Response:** 204 No Content, 26 ms, 83 B.

Bottom Screenshot: 16 - Delete Property Manager

- Method:** DELETE
- URL:** `{{base_url}}/api/property-managers/{{manager_id}}`
- Params:** Query Params table is visible.
- Response:** 204 No Content, 30 ms, 83 B.

Key	Value	Description	Bulk Edit
Key	Value	Description	

III. Redux Client

Move your React data layer to Redux Toolkit, integrating with the FastAPI + MySQL backend above.

1. Redux Setup

- Create a Redux store using Redux Toolkit.
- Define a slice to manage the state of your primary domain entity.
- Include reducers/actions for fetching, adding, updating, and deleting records.

2. Async Thunks with Axios

- Implement async thunks that call your FastAPI endpoints: fetch (GET list), create (POST), update (PUT `/id`), delete (DELETE `/id`).

- Handle success and error cases properly.

3. Home Screen

- Display all records by reading from Redux state; the list updates automatically when records are created, updated, or deleted.

4. Create / Update Screens

- Build a form to create a new record and a form to update an existing record (select by ID).
- On submission, dispatch the correct Redux thunk and show the updated results on the Home Screen.

5. Delete Functionality

- Add a delete button next to each record in the list. On click, dispatch the delete thunk and update Redux state to remove the record.

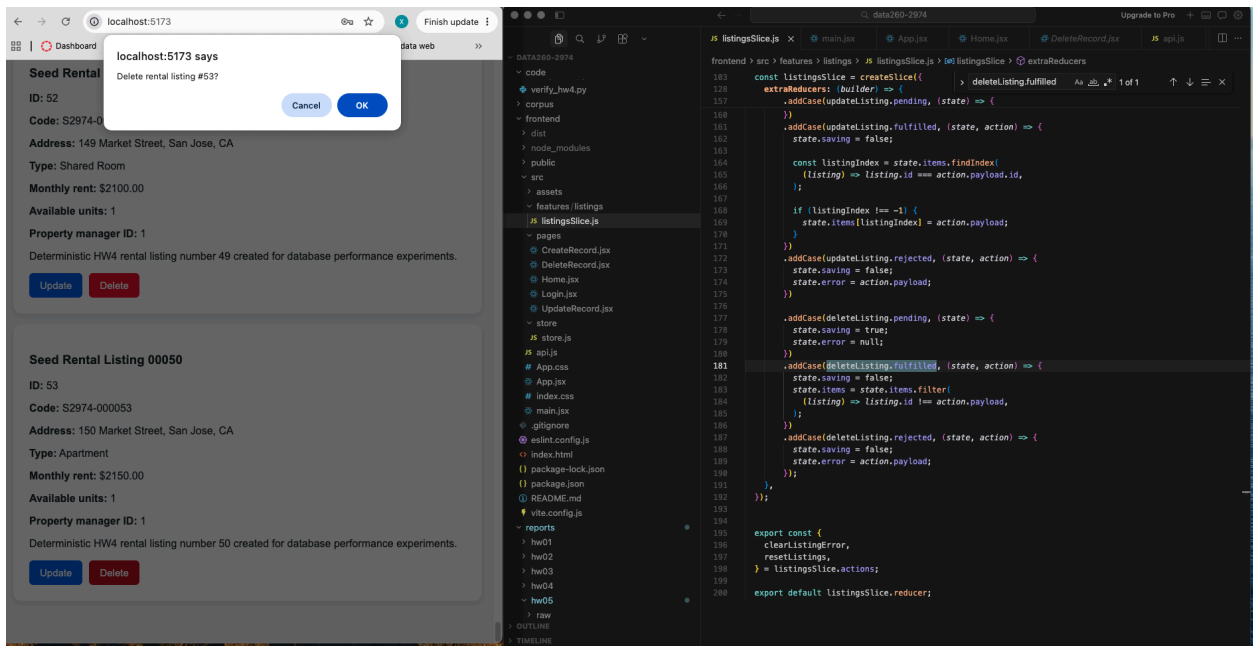
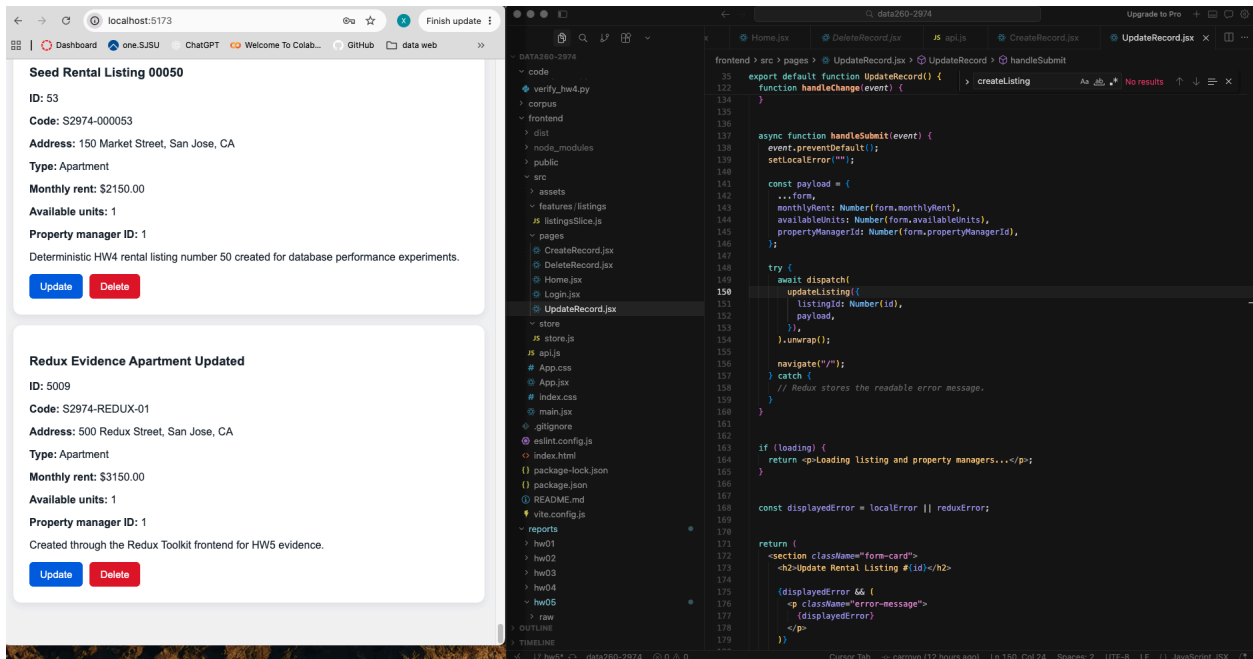
Submit one screenshot per feature (Home, Create, Update, Delete) showing the code snippet (Redux thunk or slice) and the corresponding UI output together, in the same frame. Do not submit full code - only the relevant code snippet and working result.

The first screenshot shows the Home screen of the application. It displays a list of seed rental listings. The first listing is 'Seed Rental Listing 00001' with ID 4, code S2974-000004, address 101 Market Street, San Jose, CA, type Shared Room, monthly rent \$1700.00, and 1 available unit. The second listing is 'Seed Rental Listing 00002' with ID 5, code S2974-000005, address 102 Market Street, San Jose, CA, type Apartment, monthly rent \$2150.00, and 1 available unit. The screen includes 'Add Listing' and 'Refresh' buttons.

The second screenshot shows the CreateRecord screen. It displays a form to create a new listing. The form fields are: ID (53), Code (S2974-000053), Address (150 Market Street, San Jose, CA), Type (Apartment), Monthly rent (\$2150.00), Available units (1), and Property manager ID (1). The form includes 'Update' and 'Delete' buttons.

The third screenshot shows the DeleteRecord screen. It displays a form to delete an existing listing. The form fields are: ID (5009), Code (S2974-REDUX-01), Address (500 Redux Street, San Jose, CA), Type (Apartment), Monthly rent (\$3100.00), Available units (2), and Property manager ID (1). The form includes 'Update' and 'Delete' buttons.

The code snippets show the Redux thunks for creating, updating, and deleting a listing. The 'createRecord' thunk dispatches 'createListing' and 'setLocalError'. The 'updateRecord' thunk dispatches 'updateListing' and 'setLocalError'. The 'deleteRecord' thunk dispatches 'deleteListing' and 'setLocalError'.



Part 2 - MCP Tool Servers

A. TheMealDB Tutorial Server

Build a local MCP server in Python that provides four tools for searching for and retrieving recipes from the public TheMealDB API. You will run and debug the server using the MCP Inspector.

Connecting it to a desktop MCP client is optional and is not required for credit.

You do not need to create an API key for this assignment. TheMealDB allows a test key of 1 for education/dev, and it publishes simple JSON endpoints. (themealdb.com)

What you'll build

A local MCP server named meals that provides four tools:

6. `search_meals_by_name(query, limit=5)`

- Call TheMealDB Search meal by name endpoint: `.../search.php?s=<query>`
- Return up to limit meals with: name, area (cuisine), category, thumbnail, and the meal id for drill-down.

7. `meals_by_ingredient(ingredient, limit=12)`

- Call Filter by main ingredient: `.../filter.php?i=<ingredient>`
- Return small cards (name, thumbnail, id).

8. `random_meal()`

- Call Lookup a single random meal: `.../random.php`.

9. `meal_details(id)`

Call Lookup a single meal by ID and return its full recipe details.

The API base is `https://www.themealdb.com/api/json/v1/1/...` and the published "test key" is 1, which is sufficient for this homework.

Set up your environment

10. Install the MCP Python SDK with CLI: `pip install "mcp[cli]"`

11. Write a single Python file (e.g., `meals_server.py`) that defines your tools and runs FastMCP over STDIO.

Important logging rule: For STDIO servers, never write to stdout (that corrupts the JSONRPC stream). Log to stderr using a logger.

I/O - Input → Output shapes you should implement:

12. `search_meals_by_name(query, limit)`

- Input: query: str, limit: int (1–25)
- Output (list): objects with { id, name, area, category, thumb }

13. `meals_by_ingredient(ingredient, limit)`

- Input: ingredient: str, limit: int
- Output (list): objects with { id, name, thumb }

14. `meal_details(id)`

- Input: id: str | int

- Output (object): { id, name, category, area, instructions, image, source, youtube, ingredients: [{name, measure}] }

15. random_meal()

- Input: none

- Output: same shape as meal_details.

- Error handling:

if an endpoint returns "meals": null (no results), return a clear, empty result with a short message (e.g., "no matches"). Network or JSON errors should raise a clean error to the client (MCP Inspector will display it).

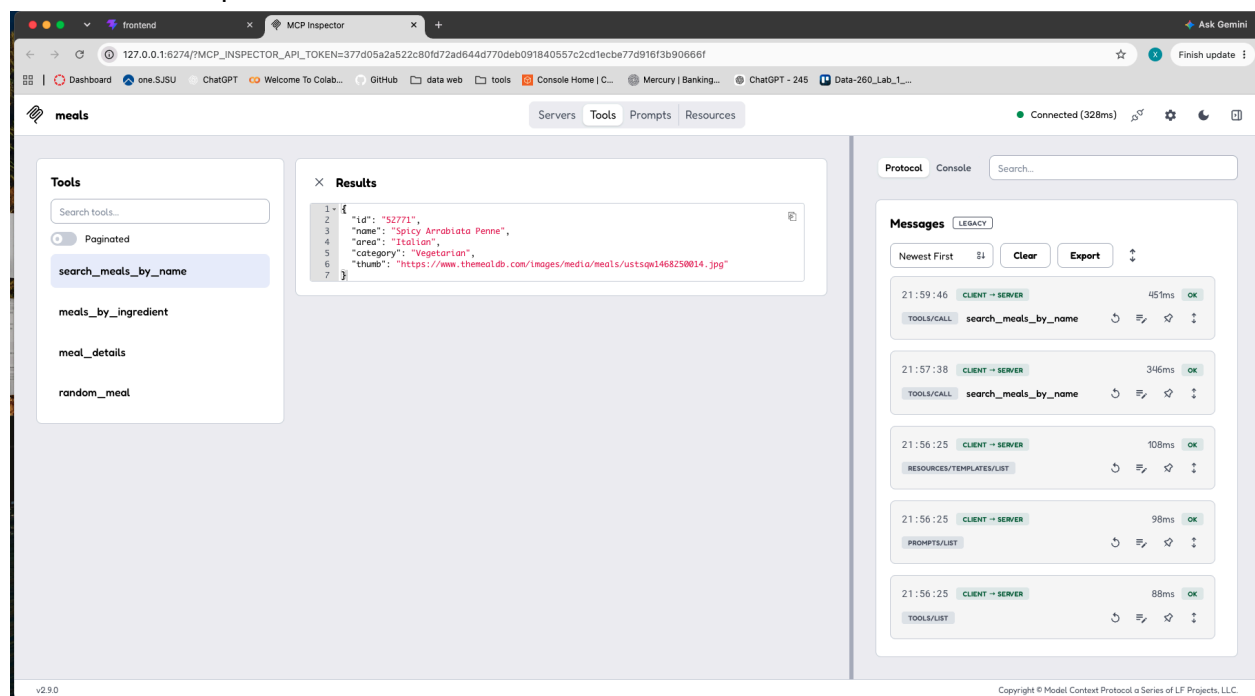
Run and debug locally with MCP Inspector

16. In your project folder, open a terminal and start development mode with the server file you wrote: `mcp dev <your_file_name.py>`

17. The Inspector opens in your browser. Use it to call each tool with test inputs (e.g., "Arrabiata",

ingredient "chicken"), verify outputs look like your designed schema, and fix any errors before moving on.

Include one labeled MCP Inspector screenshot for each of the four tools, showing the input and returned output.



meals

ServersToolsPromptsResources

Connected (328ms)

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

meal_details

random_meal

Results

1

{

"id": "52940",

"name": "Brown Stew Chicken",

"thumb": "https://www.themeadb.com/images/media/meals/syxp151536095.jpg"

}

2

3

4

5

1

{

"id": "53161",

"name": "Chicken & chorizo rice pot",

"thumb": "https://www.themeadb.com/images/media/meals/fk80jp1763280767.jpg"

}

2

3

4

5

1

{

"id": "52846",

"name": "Chicken & mushroom Hotpot",

"thumb": "https://www.themeadb.com/images/media/meals/uuspp151297945.jpg"

}

2

3

4

5

Protocol Console

Search...

Messages

LEGACY

Newest First

Clear

Export

22:01:14

CLIENT - SERVER

259ms

OK

TOOLS/CALL

meals_by_ingredient

21:59:46

CLIENT - SERVER

451ms

OK

TOOLS/CALL

search_meals_by_name

21:57:38

CLIENT - SERVER

346ms

OK

TOOLS/CALL

search_meals_by_name

21:56:25

CLIENT - SERVER

108ms

OK

RESOURCES/TEMPLATES/LIST

21:56:25

CLIENT - SERVER

98ms

OK

PROMPTS/LIST

v2.9.0

Copyright © Model Context Protocol a Series of LF Projects, LLC.

meals

ServersToolsPromptsResources

Connected (328ms)

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

meal_details

random_meal

Results

1

{

"id": "53217",

"name": "Shawarma chuck roast wrap",

"category": "Beef",

"area": "Saudi Arabian",

"instructions": "In a dutch oven or slow cooker, add all the spices, beef broth and 1",

"image": "https://www.themeadb.com/images/media/meals/swa87v1763595282.jpg",

"source": "https://cookpad.com/eng/recipes/2441590?ref=search&search_term=saudi+arabia",

"youtube": "https://www.youtube.com/shorts/7LhyRjyKcE",

"ingredients": [

{

"name": "Chuck Roast",

"measure": "1kg"

},

{

"name": "Beef Stock",

"measure": "1 cup"

},

{

"name": "Lemon Juice",

"measure": "3 tablespoons"

},

{

"name": "Ground Cumin",

"measure": "1 tablespoon"

},

{

"name": "Paprika",

"measure": "2 tsp"

},

{

"name": "Salt",

"measure": "1 tsp"

},

{

"name": "Ground Ginger",

"measure": "1 tsp"

},

{

"name": "Turmeric",

"measure": "1 tsp"

},

]

}

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

Protocol Console

Search...

Messages

LEGACY

Newest First

Clear

Export

22:01:57

CLIENT - SERVER

260ms

OK

TOOLS/CALL

random_meal

22:01:14

CLIENT - SERVER

259ms

OK

TOOLS/CALL

meals_by_ingredient

21:59:46

CLIENT - SERVER

451ms

OK

TOOLS/CALL

search_meals_by_name

21:57:38

CLIENT - SERVER

346ms

OK

TOOLS/CALL

search_meals_by_name

21:56:25

CLIENT - SERVER

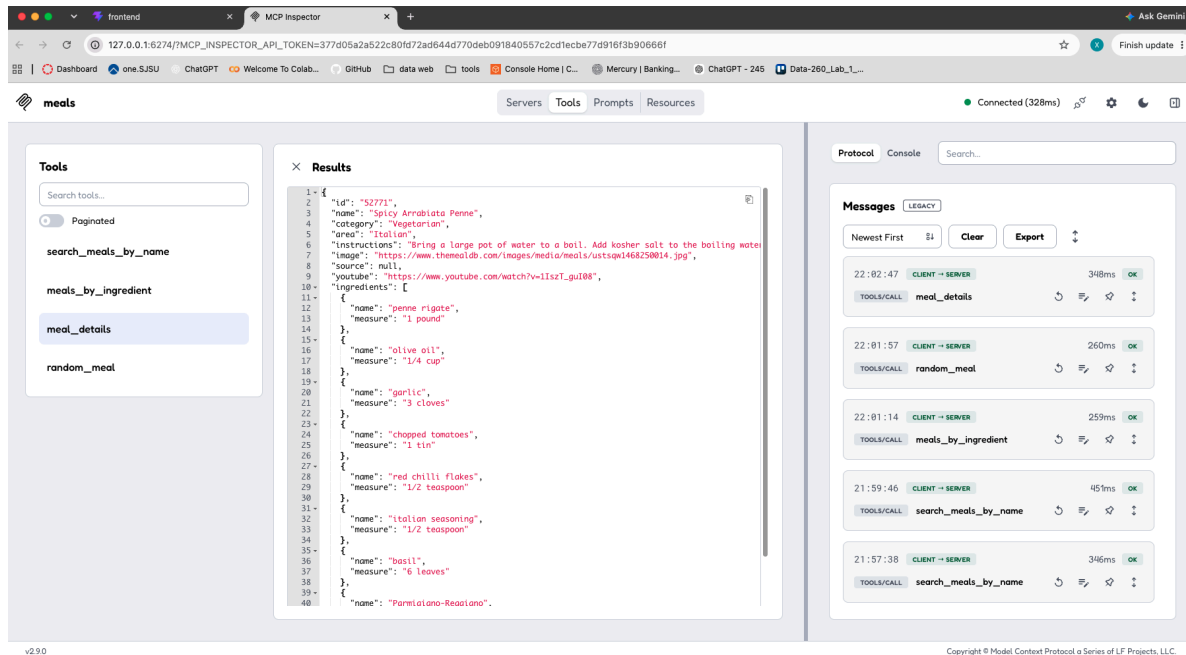
108ms

OK

RESOURCES/TEMPLATES/LIST

v2.9.0

Copyright © Model Context Protocol a Series of LF Projects, LLC.



B. Domain MCP Server (required)

- Build a second MCP server over your own <PREFIX>_rel domain data with exactly three tools: search, detail lookup, and one aggregate operation.
- Use one response envelope for all three tools: {ok, data, error}, where error is null on success. This is the same envelope shape you will reuse for execute_tool in Part 4 - define it once here and carry it through.
- All MCP STDIO logs go to stderr, never stdout.
- Validate all three domain tools in MCP Inspector with at least one successful call and one intentionally invalid call per tool. Include screenshots of both - these rejected-call examples are the ones you'll reference again in Part 3

Dashboard

one.SJSU

ChatGPT

Welcome To Colab...

GitHub

data web

tools

Console Home | C...

Mercury | Banking...

ChatGPT - 245

Data-260_Lab_1...

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=b208d7f2357ce93d6e12b48fcaaac46e0b4f57dcb5490e7d792e4dcf0578c3

☆

⬇

⌵

Finish update

s2974_rel

Servers

Tools

Prompts

Resources

Connected (520ms)

⌵

⚙

🌙

🖨

Tools

Search tools...

Paginated

search_listings

listing_details

manager_rent_summary

Results

1 {

2 "ok": false,

3 "data": null,

4 "error": "limit must be an integer between 1 and 25"

5 }

Protocol

Console

Search...

Messages

LEGACY

Newest First

⌵

Clear

Export

⌵

22:48:34

CLIENT → SERVER

45ms

OK

TOOLS/CALL

search_listings

⌵

⌵

⌵

⌵

22:47:47

CLIENT → SERVER

119ms

OK

TOOLS/CALL

search_listings

⌵

⌵

⌵

⌵

22:47:15

CLIENT → SERVER

124ms

OK

RESOURCES/TEMPLATES/LIST

⌵

⌵

⌵

⌵

22:47:15

CLIENT → SERVER

114ms

OK

PROMPTS/LIST

⌵

⌵

⌵

⌵

22:47:15

CLIENT → SERVER

105ms

OK

TOOLS/LIST

⌵

⌵

⌵

⌵

v2.9.0

Copyright © Model Context Protocol a Series of LF Projects, LLC.

MCP Inspector

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=b208df72357ce93d62e12b48fcaaac46e0b4f57dcb5490e7d792e4dcf0578c3

Dashboardone.SJSUChatGPTWelcome To Colab...GitHubdata webtoolsConsole Home | C...Mercury | Banking...ChatGPT - 245Data-260_Lab_1...

s2974_rel

ServersToolsPromptsResources

Connected (520ms)

Tools

Search tools...

Paginated

search_listings

listing_details

manager_rent_summary

Results

```
1 {
2   "ok": true,
3   "data": {
4     "id": 4,
5     "listingCode": "52974-000004",
6     "listingTitle": "Seed Rental Listing 00001",
7     "address": "101 Market Street, San Jose, CA",
8     "submitterEmail": "seed-owner-00001@example.com",
9     "description": "Deterministic HW4 rental listing number 1 created",
10    "propertyType": "Shared Room",
11    "monthlyRent": "1700.00",
12    "availableUnits": 1,
13    "termsAccepted": true,
14    "propertyManagerId": 1,
15    "created_at": "2026-09-27T12:14:44",
16    "updated_at": "2026-10-03T23:05:36",
17    "propertyManager": {
18      "id": 1,
19      "firstName": "Default",
20      "lastName": "Manager",
21      "email": "manager2974@example.com"
22    }
23  },
24  "error": null
25 }
```

ProtocolConsole

Search...

Messages

LEGACY

Newest First

Clear

Export

22:49:09 CLIENT → SERVER 41ms OK

TOOLS/CALL listing_details

22:48:34 CLIENT → SERVER 45ms OK

TOOLS/CALL search_listings

22:47:47 CLIENT → SERVER 119ms OK

TOOLS/CALL search_listings

22:47:15 CLIENT → SERVER 124ms OK

RESOURCES/TEMPLATES/LIST

22:47:15 CLIENT → SERVER 114ms OK

PROMPTS/LIST

v2.9.0

Copyright © Model Context Protocol a Series of LF Projects, LLC.

MCP Inspector

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=b208df72357ce93d62e12b48fcaaac46e0b4f57dcb5490e7d792e4dcf0578c3

Dashboardone.SJSUChatGPTWelcome To Colab...GitHubdata webtoolsConsole Home | C...Mercury | Banking...ChatGPT - 245Data-260_Lab_1...

s2974_rel

ServersToolsPromptsResources

Connected (520ms)

Tools

Search tools...

Paginated

search_listings

listing_details

manager_rent_summary

Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "listing_id must be a positive integer"
5 }
```

ProtocolConsole

Search...

Messages

LEGACY

Newest First

Clear

Export

22:49:48 CLIENT → SERVER 44ms OK

TOOLS/CALL listing_details

22:49:09 CLIENT → SERVER 41ms OK

TOOLS/CALL listing_details

22:48:34 CLIENT → SERVER 45ms OK

TOOLS/CALL search_listings

22:47:47 CLIENT → SERVER 119ms OK

TOOLS/CALL search_listings

22:47:15 CLIENT → SERVER 124ms OK

RESOURCES/TEMPLATES/LIST

v2.9.0

Copyright © Model Context Protocol a Series of LF Projects, LLC.

MCP Inspector

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=b208df72357ce93d62e12b48fcaaac46e0b4f57dcb5490e7d792e4dcf0578c3

Dashboard one.SJSU ChatGPT Welcome To Colab... GitHub data web tools Console Home | C... Mercury | Banking... ChatGPT - 245 Data-260_Lab_1...

s2974_rel Servers Tools Prompts Resources

Connected (520ms)

Tools

Search tools...

Paginated

search_listings

listing_details

manager_rent_summary

Results

```
1- {
2-   "ok": true,
3-   "data": {
4-     "propertyManager": {
5-       "id": 1,
6-       "firstName": "Default",
7-       "lastName": "Manager",
8-       "email": "manager2974@example.com"
9-     },
10-    "listingCount": 5000,
11-    "totalAvailableUnits": 5000,
12-    "averageRent": "1975.00",
13-    "minimumRent": "1500.00",
14-    "maximumRent": "2450.00"
15-  },
16-   "error": null
17- }
```

Protocol Console Search...

Messages LEGACY

Newest First Clear Export

22:50:13 CLIENT - SERVER 54ms OK
TOOLS/CALL manager_rent_summary

22:49:48 CLIENT - SERVER 44ms OK
TOOLS/CALL listing_details

22:49:09 CLIENT - SERVER 41ms OK
TOOLS/CALL listing_details

22:48:34 CLIENT - SERVER 45ms OK
TOOLS/CALL search_listings

22:47:47 CLIENT - SERVER 119ms OK
TOOLS/CALL search_listings

v2.9.0 Copyright © Model Context Protocol a Series of LF Projects, LLC.

MCP Inspector

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=b208df72357ce93d62e12b48fcaaac46e0b4f57dcb5490e7d792e4dcf0578c3

Dashboard one.SJSU ChatGPT Welcome To Colab... GitHub data web tools Console Home | C... Mercury | Banking... ChatGPT - 245 Data-260_Lab_1...

s2974_rel Servers Tools Prompts Resources

Connected (520ms)

Tools

Search tools...

Paginated

search_listings

listing_details

manager_rent_summary

Results

```
1- {
2-   "ok": false,
3-   "data": null,
4-   "error": "property_manager_id must be a positive integer"
5- }
```

Protocol Console Search...

Messages LEGACY

Newest First Clear Export

22:50:44 CLIENT - SERVER 52ms OK
TOOLS/CALL manager_rent_summary

22:50:13 CLIENT - SERVER 54ms OK
TOOLS/CALL manager_rent_summary

22:49:48 CLIENT - SERVER 44ms OK
TOOLS/CALL listing_details

22:49:09 CLIENT - SERVER 41ms OK
TOOLS/CALL listing_details

22:48:34 CLIENT - SERVER 45ms OK
TOOLS/CALL search_listings

v2.9.0 Copyright © Model Context Protocol a Series of LF Projects, LLC.

Part 3 - Tool Contracts Under Stress

18. Using the same three domain tools and the same rejected-call examples you captured in Part 2B, Write down expected JSON input schema, the rejected JSON input, and the returned error output for each call. Documents why each Part 2B call was rejected - you do not need to re-run new rejected calls here.

Tool	Expected Input	Rejected Input	Returned Error	Reason
search_listings	query must be a non-empty string; limit must be an integer from 1 to 25.	{"query": "Apartment", "limit": 0}	{"ok": false, "data": null, "error": "limit must be an integer between 1 and 25"}	The call was rejected because the limit was outside the allowed range.
listing_details	listing_id must be a positive integer.	{"listing_id": 0}	{"ok": false, "data": null, "error": "listing_id must be a positive integer"}	The call was rejected because zero is not a valid listing ID.
manager_rent_summary	property_manager_id must be a positive integer.	{"property_manager_id": 0}	{"ok": false, "data": null, "error": "property_manager_id must be a positive integer"}	The call was rejected because zero is not a valid property-manager ID.

19. Add timeouts and a bounded exponential-backoff retry policy to API or storage operations that may fail. Demonstrate (Show on report):

- Success on the first attempt.
- Failure on the first attempt followed by success after a retry.
- Failure after all allowed retries, returning a clean error result instead of crashing.

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % mkdir -p reports/hw05/screenshots/part3

python -m py_compile code/hw5_retry_demo.py
python code/hw5_retry_demo.py
Student: Xuanhua Li | SID4: 2974
Timestamp: 2026-10-04T16:14:51.595695-07:00

SCENARIO 1: SUCCESS ON FIRST ATTEMPT
Result:
{
  "ok": true,
  "data": {
    "id": 4,
    "listingCode": "S2974-000004",
    "listingTitle": "Seed Rental Listing 00001",
    "address": "101 Market Street, San Jose, CA",
    "submitterEmail": "seed-owner-00001@example.com",
    "description": "Deterministic HW4 rental listing number 1 created for database performance experiments"
  },
  "propertyType": "Shared Room",
  "monthlyRent": "1700.00",
  "availableUnits": 1,
  "termsAccepted": true,
  "propertyManagerId": 1,
  "created_at": "2026-09-27T12:14:44",
  "updated_at": "2026-10-03T23:05:36",
  "propertyManager": {
    "id": 1,
    "firstName": "Default",
    "lastName": "Manager",
    "email": "manager2974@example.com"
  }
},
  "error": null
}
Retry trace:
[
  {
    "attempt": 1,
    "status": "success",
    "latency_ms": 22.535
  }
]

```

```

SCENARIO 2: FIRST ATTEMPT FAILS, SECOND SUCCEEDS
Result:
{
  "ok": true,
  "data": {
    "id": 4,
    "listingCode": "S2974-000004",
    "listingTitle": "Seed Rental Listing 00001",
    "address": "101 Market Street, San Jose, CA",
    "submitterEmail": "seed-owner-00001@example.com",
    "description": "Deterministic HW4 rental listing number 1 created for database performance experiments"
  },
  "propertyType": "Shared Room",
  "monthlyRent": "1700.00",
  "availableUnits": 1,
  "termsAccepted": true,
  "propertyManagerId": 1,
  "created_at": "2026-09-27T12:14:44",
  "updated_at": "2026-10-03T23:05:36",
  "propertyManager": {
    "id": 1,
    "firstName": "Default",
    "lastName": "Manager",
    "email": "manager2974@example.com"
  }
},
  "error": null
}
Retry trace:
[
  {
    "attempt": 1,
    "status": "failure",
    "error": "injected failure on attempt 1",
    "latency_ms": 0.002
  },
  {
    "attempt": 2,
    "status": "success",
    "latency_ms": 0.973
  }
]

```

```

SCENARIO 3: ALL ATTEMPTS FAIL
Result:
{
  "ok": false,
  "data": null,
  "error": "database operation failed after 3 attempts"
}
Retry trace:
[
  {
    "attempt": 1,
    "status": "failure",
    "error": "injected failure on attempt 1",
    "latency_ms": 0.001
  },
  {
    "attempt": 2,
    "status": "failure",
    "error": "injected failure on attempt 2",
    "latency_ms": 0.007
  },
  {
    "attempt": 3,
    "status": "failure",
    "error": "injected failure on attempt 3",
    "latency_ms": 0.007
  }
]
RETRY DEMONSTRATION: 3/3 PASS

```

20. Use VERIFY_SEED to reproducibly simulate failures at rates of 0%, 20%, and 50%. The same seed must produce the same success/failure sequence each time. Run 50 calls at each failure rate, for 150 calls total. For each rate, report the success rate, average latency, and p99 latency. Save the raw data for all 150 calls in reports/hw05/raw/, so your results can be Double-checked.

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % python -m py_compile code/hw5_failure_experiment.py
python code/hw5_failure_experiment.py
Student: Xuanhua Li | SID4: 2974
VERIFY_SEED: 262974
Retry policy: 3 attempts, 1.0s timeout, 10ms/20ms bounded backoff

RATE   CALLS   SUCCESS RATE   MEAN LATENCY   P99 LATENCY
-----
0%      50     100.00%       0.889 ms      20.232 ms
20%     50     96.00%        4.462 ms      38.206 ms
50%     50     94.00%       10.929 ms      39.003 ms

RAW CALL RECORDS: 150
DETERMINISM CHECK: PASS
FAILURE-INJECTION EXPERIMENT: PASS
○ ((.venv) ) carol@Carols-MacBook-Air data260-2974 %

```

Student: Xuanhua Li | SID4: 2974

VERIFY_SEED: 262974

Retry policy: 3 attempts, 1.0s timeout, 10ms/20ms bounded backoff

RATE CALLS SUCCESS RATE MEAN LATENCY P99 LATENCY

```

-----
0%      50     100.00%       0.889 ms      20.232 ms
20%     50     96.00%        4.462 ms      38.206 ms
50%     50     94.00%       10.929 ms      39.003 ms

```

RAW CALL RECORDS: 150
DETERMINISM CHECK: PASS
FAILURE-INJECTION EXPERIMENT: PASS

21. Use your results table below to evaluate whether the retry policy is suitable for an interactive assistant. Then explain how you would modify the number of retries, timeout, or backoff delays for batch processing, where longer processing times are acceptable.

The current retry policy is suitable for an interactive assistant in this controlled experiment. Across the three simulated failure rates, the observed success rates were 100%, 96%, and 94%, while p99 latency remained below 40 ms. The three-attempt limit prevents requests from retrying indefinitely, and the tool returns a controlled error when all attempts fail. This gives the assistant a reasonable balance between recovery and response time.

These latency results come from a controlled failure-injection experiment and should not be treated as a guarantee for a real network or production database.

For batch processing, I would allow more retry attempts, increase the per-attempt timeout, and use longer bounded exponential-backoff delays with jitter. For example, a batch job could use five attempts, a three-to-five-second timeout, and progressively longer delays. Batch workloads can tolerate longer completion times, so eventual success is more important than immediate response. However, validation errors and other non-transient failures should still return immediately because retrying them would not resolve the problem.

Part 4 - One Safe Tool Entry Point and Offline Tests

22. Create `execute_tool(name, inputs)`. This is the single function that the Part 5 agent will use to call any of the three domain tools. It must reuse the {ok, data, error} format from Part 2B, return the result as a JSON string, and handle errors without crashing.

23. Write a simple Python test runner using assert statements. For each of the three domain tools, test at least one valid input and the invalid input cases described in Part 3. Include at least six tests overall. Print PASS or FAIL for each test and a final X/Y summary.

24. Use temporary fixtures or dependency injection so the tests run offline, repeatedly, and without an LLM.

Include one screenshot of the terminal running the Part 4 test runner. The screenshot must show the PASS/FAIL result for each test and the final X/Y summary.

```
● ((.venv) ) carol@Carols-MacBook-Air data260-2974 % mkdir -p reports/hw05/screenshots/part4

python -m py_compile \
  code/tool_executor.py \
  code/test_hw5_offline.py

python code/test_hw5_offline.py
Student: Xuanhua Li | SID4: 2974
HW5 PART 4 OFFLINE TOOL TESTS
Database: temporary in-memory SQLite fixture
LLM: not used

PASS | search valid
PASS | search invalid
PASS | detail valid
PASS | detail invalid
PASS | aggregate valid
PASS | aggregate invalid
PASS | unknown tool
PASS | non-object inputs

FINAL SUMMARY: 8/8 PASS
○ ((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

Part 5 - Agent Loop, Safety Rule, and Reflection

This part adds a business-logic safety rule and connects the tested tool layer to an Ollama-based agent.

I. Safety rule

- Add one domain-specific safety rule inside `execute_tool`.
- When the rule is violated, return `{ok: false, data: null, error: "..."} without raising an exception.`
- Demonstrate one allowed call and one blocked call.

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % mkdir -p reports/hw05/screenshots/part5

python -m py_compile \
code/tool_executor.py \
code/hw5_safety_demo.py

python code/hw5_safety_demo.py
Student: Xuanhua Li | SID4: 2974
Timestamp: 2026-10-04T19:41:16.470376-07:00

ALLOWED CALL
{
  "input": {
    "query": "Apartment",
    "limit": 1
  },
  "ok": true,
  "error": null
}

BLOCKED CALL
{
  "input": {
    "query": "no families with children",
    "limit": 5
  },
  "ok": false,
  "data": null,
  "error": "Safety rule blocked a discriminatory housing search."
}

SAFETY RULE DEMONSTRATION: 2/2 PASS
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

II. Agent loop

- Implement `run_agent(user_input)` using the local Ollama model and the three domain tools.
- Add a turn counter and `max_steps` limit.
- Log each step, tool call, input, result, and final stop reason to `agent_runs.jsonl`.

III. Additional tests

After implementing the Part 5 safety rule and agent loop, add two tests to the existing Part 4 offline test suite. Test that `execute_tool` blocks a call that violates the safety rule, and that `run_agent`, using `MockModel`, stops after reaching `max_steps`. Both tests must run offline without an API key, live model, external API, or database.

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % python -m py_compile \
code/tool_executor.py \
code/hw5_agent.py \
code/test_hw5_offline.py

python code/test_hw5_offline.py
Student: Xuanhua Li | SID4: 2974
HW5 PART 4 + PART 5 OFFLINE TESTS
Database: temporary in-memory SQLite fixture
LLM: mocked; Ollama is not called

PASS | search valid
PASS | search invalid
PASS | detail valid
PASS | detail invalid
PASS | aggregate valid
PASS | aggregate invalid
PASS | unknown tool
PASS | non-object inputs
PASS | safety rule block
PASS | agent max steps

FINAL SUMMARY: 10/10 PASS
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % python code/hw5_agent.py \
  "Find two apartment rental listings." \
  --max-steps 4
{
  "run_id": "7c8bba7b1f204447831fde194cc92509",
  "timestamp": "2026-10-04T19:54:19.982526-07:00",
  "model": "qwen3:8b",
  "user_input": "Find two apartment rental listings.",
  "max_steps": 4,
  "steps": [
    {
      "step": 1,
      "model_message": "",
      "tool_calls": [
        {
          "name": "search_listings",
          "inputs": {
            "query": "apartment",
            "limit": 2
          },
          "result": {
            "ok": true,
            "data": [
              {
                "id": 5,
                "listingCode": "S2974-000005",
                "listingTitle": "Seed Rental Listing 00002",
                "address": "102 Market Street, San Jose, CA",
                "propertyType": "Apartment",
                "monthlyRent": "1750.00",
                "availableUnits": 1,
                "propertyManagerId": 1
              },
              {
                "id": 9,
                "listingCode": "S2974-000009",
                "listingTitle": "Seed Rental Listing 00006",
                "address": "106 Market Street, San Jose, CA",
                "propertyType": "Apartment",
                "monthlyRent": "1950.00",
                "availableUnits": 1,
                "propertyManagerId": 1
              }
            ]
          },
          "error": null
        }
      ]
    }
  ]
}

```

```

      "step": 2,
      "model_message": "Here are two apartment rental listings:\n\n1. **Listing Code:** S2974-000005 \n
**Title:** Seed Rental Listing 00002 \n **Address:** 102 Market Street, San Jose, CA \n **Monthly R
ent:** $1,750.00 \n **Available Units:** 1 \n **Property Manager ID:** 1 \n\n2. **Listing Code:** S
2974-000009 \n **Title:** Seed Rental Listing 00006 \n **Address:** 106 Market Street, San Jose, CA
\n **Monthly Rent:** $1,950.00 \n **Available Units:** 1 \n **Property Manager ID:** 1",
      "tool_calls": []
    }
  ],
  "step_count": 2,
  "tool_call_count": 1,
  "stop_reason": "normal_completion",
  "final_answer": "Here are two apartment rental listings:\n\n1. **Listing Code:** S2974-000005 \n **Ti
tle:** Seed Rental Listing 00002 \n **Address:** 102 Market Street, San Jose, CA \n **Monthly Rent:**
$1,750.00 \n **Available Units:** 1 \n **Property Manager ID:** 1 \n\n2. **Listing Code:** S2974-
000009 \n **Title:** Seed Rental Listing 00006 \n **Address:** 106 Market Street, San Jose, CA \n
**Monthly Rent:** $1,950.00 \n **Available Units:** 1 \n **Property Manager ID:** 1"
}

```

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % python code/hw5_agent.py \
  "Show me the full details for rental listing ID 4." \
  --max-steps 4
{
  "run_id": "fe3691f0d11f4585bcc8d89af5c4974d",
  "timestamp": "2026-10-04T19:57:40.872633-07:00",
  "model": "qwen3:8b",

```

```

"model": "qwen3:8b",
"user_input": "Show me the full details for rental listing ID 4.",
"max_steps": 4,
"steps": [
  {
    "step": 1,
    "model_message": "",
    "tool_calls": [
      {
        "name": "listing_details",
        "inputs": {
          "listing_id": 4
        },
        "result": {
          "ok": true,
          "data": {
            "id": 4,
            "listingCode": "S2974-000004",
            "listingTitle": "Seed Rental Listing 00001",
            "address": "101 Market Street, San Jose, CA",
            "submitterEmail": "seed-owner-00001@example.com",
            "description": "Deterministic HW4 rental listing number 1 created for database performance experiments.",
            "propertyType": "Shared Room",
            "monthlyRent": "1700.00",
            "availableUnits": 1,
            "termsAccepted": true,
            "propertyManagerId": 1,
            "created_at": "2026-09-27T12:14:44",
            "updated_at": "2026-10-03T23:05:36",
            "propertyManager": {
              "id": 1,
              "firstName": "Default",
              "lastName": "Manager",
              "email": "manager2974@example.com"
            }
          }
        },
        "error": null
      }
    ]
  },
  {
    "step": 2,
    "model_message": "The full details for rental listing ID 4 are as follows:\n\n- **Listing Code:** S2974-000004\n- **Title:** Seed Rental Listing 00001\n- **Address:** 101 Market Street, San Jose, CA\n- **Submitter Email:** seed-owner-00001@example.com\n- **Description:** Deterministic HW4 rental listing number 1 created for database performance experiments.\n- **Property Type:** Shared Room\n- **Monthly Rent:** $1700.00\n- **Available Units:** 1\n- **Terms Accepted:** Yes\n- **Property Manager:** Default Manager (Email: manager2974@example.com)\n\nLast Updated: 2026-10-03T23:05:36",
    "tool_calls": []
  }
],
"step_count": 2,
"tool_call_count": 1,
"stop_reason": "normal_completion",
"final_answer": "The full details for rental listing ID 4 are as follows:\n\n- **Listing Code:** S2974-000004\n- **Title:** Seed Rental Listing 00001\n- **Address:** 101 Market Street, San Jose, CA\n- **Submitter Email:** seed-owner-00001@example.com\n- **Description:** Deterministic HW4 rental listing number 1 created for database performance experiments.\n- **Property Type:** Shared Room\n- **Monthly Rent:** $1700.00\n- **Available Units:** 1\n- **Terms Accepted:** Yes\n- **Property Manager:** Default Manager (Email: manager2974@example.com)\n\nLast Updated: 2026-10-03T23:05:36"
}

```

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % python code/hw5_agent.py \
  "Give me the rent summary for property manager ID 1." \
  --max-steps 4
{
  "run_id": "d52240973bf24afaab9e94dd0515e0bb",
  "timestamp": "2026-10-04T19:58:43.516217-07:00",
  "model": "qwen3:8b",
  "user_input": "Give me the rent summary for property manager ID 1.",
  "max_steps": 4,
  "steps": [
    {
      "step": 1,
      "model_message": "",
      "tool_calls": [
        {
          "name": "manager_rent_summary",
          "inputs": {
            "property_manager_id": 1
          },
          "result": {
            "ok": true,
            "data": {
              "propertyManager": {
                "id": 1,
                "firstName": "Default",
                "lastName": "Manager",
                "email": "manager2974@example.com"
              },
              "listingCount": 5000,
              "totalAvailableUnits": 5000,
              "averageRent": "1975.00",
              "minimumRent": "1500.00",
              "maximumRent": "2450.00"
            }
          },
          "error": null
        }
      ]
    },
    {
      "step": 2,
      "model_message": "The property manager with ID 1 has a total of 5,000 available units. The average rent is $1,975.00, with a minimum rent of $1,500.00 and a maximum rent of $2,450.00.",
      "tool_calls": []
    }
  ],
  "step_count": 2,
  "tool_call_count": 1,
  "stop_reason": "normal completion",
  "final_answer": "The property manager with ID 1 has a total of 5,000 available units. The average rent is $1,975.00, with a minimum rent of $1,500.00 and a maximum rent of $2,450.00."
}

```

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % python code/hw5_agent.py \
  "Find rental listings for no families with children." \
  --max-steps 4
{
  "run_id": "f787df82904d417c942ba6c54e3c03cd",
  "timestamp": "2026-10-04T20:00:43.013432-07:00",
  "model": "qwen3:8b",
  "user_input": "Find rental listings for no families with children.",
  "max_steps": 4,
  "steps": [
    {
      "step": 1,
      "model_message": "",
      "tool_calls": [
        {
          "name": "search_listings",
          "inputs": {
            "query": "no families with children",
            "limit": 10
          },
          "result": {
            "ok": false,
            "data": null,
            "error": "Safety rule blocked a discriminatory housing search."
          }
        }
      ]
    }
  ],
  "step_count": 1,
  "tool_call_count": 1,
  "stop_reason": "safety_block",
  "final_answer": "Safety rule blocked a discriminatory housing search."
}

```

IV. Metrics

- Run at least four scenarios using the local Ollama model.
- Record each scenario's step count, stop reason, and tool-call count in [METRICS.md](#) and in report too.
- A hosted-model comparison is optional.

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % python -m py_compile \
code/hw5_agent_metrics.py

python code/hw5_agent_metrics.py
Student: Xuanhua Li | SID4: 2974
JSONL RECORDS: 5

SCENARIO          TOOL                STEPS  CALLS  STOP              RESULT
-----
search            search_listings      2      1     normal_completion PASS
listing details   listing_details       2      1     normal_completion PASS
manager summary   manager_rent_summary  2      1     normal_completion PASS
safety block      search_listings       1      1     safety_block      PASS

AVERAGE STEPS: 1.75
AVERAGE TOOL CALLS: 1
STOP REASONS: {'normal_completion': 3, 'safety_block': 1}

LIVE AGENT SCENARIOS: 4/4 PASS
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

V. Reflection

Write a 200–300-word reflection in REFLECTION.md and report too:

pick one run from your agent_runs.jsonl-style log and explain, step by step, what the harness did and why it stopped (normal completion, safety-rule block, or max_steps ceiling)

```
# HW5 Part 5 Reflection

I chose the safety-block run with ID `f787df82904d417c942ba6c54e3c03cd`. In this run,
the user asked the agent to find rental listings for "no families with children." I
chose this example because it clearly shows how the safety rule works.

First, the Ollama model read the request and decided that it needed the
`search_listings` tool. The model created a tool call with the original search phrase
and a limit of 10. At this point, the database had not been searched yet.

Next, `run_agent` sent the tool call to `execute_tool`. This function is the only
entry point for the three rental tools. Before running the search handler, it checked
the query against the housing-safety rule. The query contained the blocked phrase "no
families," so `execute_tool` returned an error instead of searching the database. The
result had `ok` set to false, `data` set to null, and an error message explaining why
the request was blocked.

The agent recorded the step, tool input, and result in `agent_runs.jsonl`. Because the
result was a safety error, the agent stopped immediately. The final stop reason was
`safety_block`. The complete run used one step and one tool call.
```

```
This test helped me understand why a safety rule should be enforced in application code. The system prompt can tell the model what it should do, but the model can still make a bad tool call. Checking every call inside `execute_tool` prevents the unsafe request from reaching the database.
```

AI Use Statement

1. How I Used AI

I used an AI assistant to help organize the homework requirements, review code structure, explain unfamiliar concepts, and suggest debugging steps. I integrated and adjusted the code in my repository, ran the services and experiments, tested the API and UI, collected the screenshots, and checked the final results myself.

2. Output That Required Verification

One early suggestion used a newer MCP package version that conflicted with the FastAPI and Starlette versions already used by the project.

3. How I Verified It

I checked the installed package versions, ran ``pip check``, tested the FastMCP import, and ran the existing FastAPI application again. These checks showed which dependency versions were compatible.

4. What I Changed

I used a compatible MCP version and kept the existing FastAPI, Starlette, and Uvicorn versions. After the change, ``pip check`` reported no broken requirements, FastAPI continued to run, and both MCP servers passed their tool tests.